



- Understanding Machine Learning Algorithms: Handwritten Notes -

Table of Contents

1. What is Machine Learning?
2. What are the Types of Machine Learning?
3. Supervised Machine Learning
4. Unsupervised Machine Learning
5. Reinforcement Learning
6. Semi-Supervised Learning
7. Steps in ML Project
8. Exploring Step 1 Data Collection
9. Exploring Step 2 Data Preparation
 - Exploratory Data Analysis
 - Data Preprocessing
 - Feature Engineering

Join Our Telegram Channel to Learn AI & ML: <https://lnkd.in/gEpetzaw>

Notes by RaviTeja G



10. Exploring Step 3 - Train Model on Dataset

- Types of Learning
- Under Fitting and OverFitting
- Regularization techniques
- Hyperparameter Tuning

11. Exploring Step 4 - Evaluation of a Model

- Evaluation Metrics
- Confusion Matrix
- Recall/Sensitivity
- Precision
- Specificity
- F1 Score
- AUC and ROC Curve
- Analysis of a Model

12. Supervised Learning

- Linear Regression
- Regularization Techniques
- Logistic Regression
- Decision Trees
- Ensemble Techniques
- Random Forests
- AdaBoost
- Gradient Boost
- XG Boost
- K-Nearest Neighbours
- Support Vector Machines
- Naive Bayes Classifiers

13. Unsupervised Learning

- Clustering Techniques
- K-Means Clustering
- Hierarchical Clustering
- DB Scan Clustering
- Evaluation of Clustering Models
- Curse of Dimensionality
- Principal Component Analysis

14. Cheat Sheet of Supervised and Unsupervised Algorithms

Machine Learning

Introduction:-

→ At a high-level, machine learning is simply the study of teaching a computer program/algorithm how to progressively improve upon a set task that it is given.

So, more practically it is the study of how to build application that exhibit this, Iterative Improvement.

There are many ways to implement it and largely there are 3 major categories.

- 1) supervised Learning.
- 2) unsupervised Learning.
- 3) Reinforcement Learning.

And also there are semi-supervised Learning.

Join Our Telegram Channel to Learn AI & ML:

<https://t.me/epetizaw>

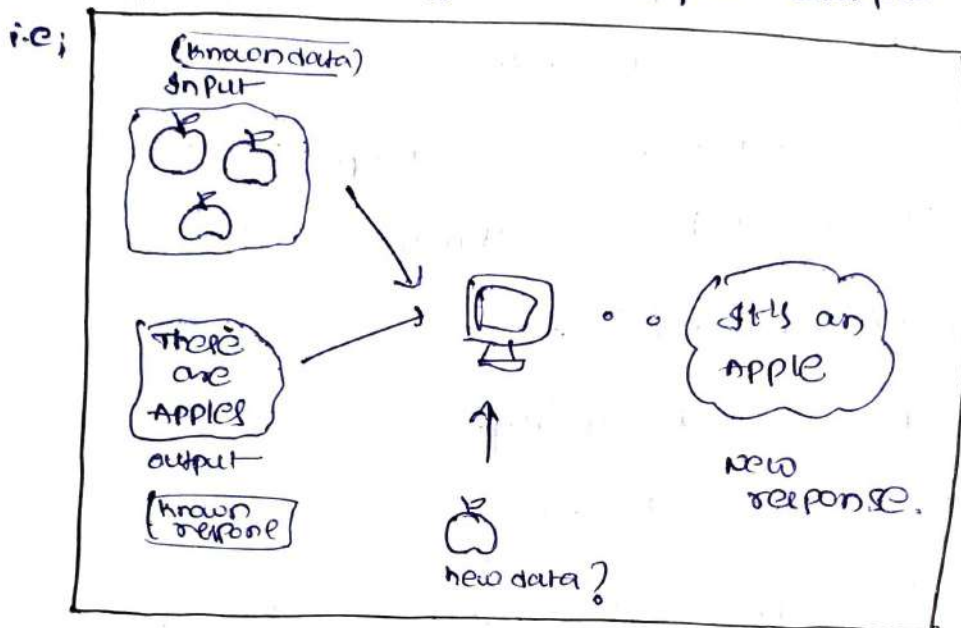
machine learning algorithms are the 'engine of machine learning', meaning it is the algorithms that turn a dataset into a model.

→ which kind of algorithm works best and what to choose (supervised, unsupervised, classification, regression, etc...) depends on the kind of problem you're solving, the computing resources available & the nature of the data.

Briefing the types:-

① Supervised Learning

Supervised learning is the most popular paradigm for performing machine learning operations. It is widely used for data where there is a precise mapping b/w input-output data.



- Given data in the form of examples with labels, we can feed a learning algorithm these 'example-label' pairs one by one. Over time, the algorithm will learn to approximate the exact nature of the relationship b/w examples & their labels. (It's called training the data)
- When fully trained, the supervised learning algorithm will be able to observe a new, never-before seen example & predict a good label for it.

→ And so supervised algorithms are called

'task-oriented'. As we provide it with more & more examples, it is able to learn more & more properly so that it can undertake the task & yield us the output more accurately.

It is exhibited in many of following applications:
Face Recognition—

Face lock app in our phone has been using a supervised learning algorithm that it is trained to recognize your face. Having a system that takes a photo, finds faces & guesses who that is in the photo (suggesting a tag) is a supervised process.

Spam classification—

Spam filter is a supervised learning system. Fed email examples & labels (spam/not-spam). These systems learn how to filter out malicious emails so that the user is not harassed.

Advertisement popularity—

Selecting advertisements that will perform well is often a supervised learning task. Many of ads you see as you browse are placed there because a learning algorithm said that they were of reasonable popularity (more clicked).

and many examples... (3)

Types of Supervised Learning

- 1) Regression
- 2) Classification
- 3) neural networks
- 4) Boosting techniques.

Regression

→ Regression algorithms predict a 'continuous value' based on the input variables.

Regression algorithms

- 1) Linear Regression
- 2) Polynomial Regression
- 3) Exponential Regression
- 4) Logarithmic Regression

Applications

- ① Risk Assessment
- ② Score Prediction etc.

Classification

→ we use classification algorithms for predict set of items class or category.

Classification algorithms

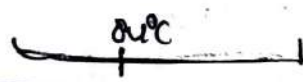
Logistic Regression

- 1) k-nearest neighbours
- 2) Decision trees
- 3) Random forest
- 4) Support vector machine (SVM)
- 5) Naive Bayes
- 6) Ada Boost
- 7) XG Boost

Applications

- ① Fraud detection
- ② Email spam detection
- ③ Image classification
- ④ Diagnosis etc.

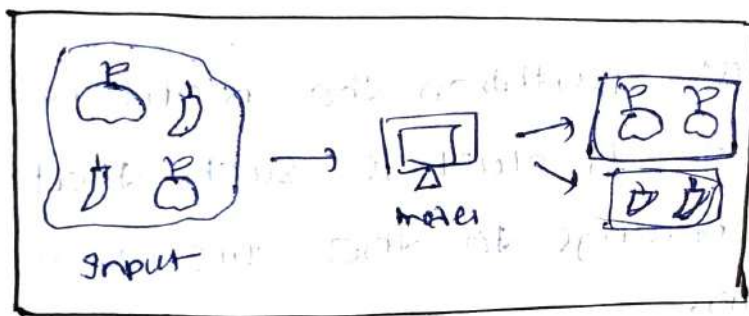
Hot	Cold
-----	------

④  temp predict

⑧ unsupervised learning

[based on Hebbian learning]

- To easy understand, in case of unsupervised learning algorithm, the data is not explicitly labeled into diff. classes, that is no labels. The model is able to learn from the data by finding implicit patterns.
- unsupervised learning algorithms identify the data based on their density / structures, similar segments & other similar features.



Types of unsupervised learning

- 1) clustering
- 2) dimensionality reduction & visualization
- 3) Anomaly detection
- 4) Association
- 5) Auto encoder

1) clustering

clustering, also known as cluster analysis, is a technique of grouping similar sets of objects in the same group that is different from the objects in other groups.

→ Some of the essential clustering techniques are as follows:

a) K-means

In this we partition the n observations in the data into K clusters such that each observation belongs to the cluster with nearest mean.

b) DBSCAN

It groups the data based on density. It groups together the points that are given in space & marks the outliers in low density regions.

c) Hierarchical clustering

A hierarchy of clusters is built.

d) mean shift

2) Dimensionality Reduction & Visualization

a) Principal Component Analysis (PCA)

Reduce data from more dimensions to lower dimensions while attempting to preserve the variance.

→ we reduce the size of data to extract useful information.

b) t-Distributed Stochastic Neighbor Embedding (t-SNE)

It is used to visualize high-dimensional data in a 2D/3D space.

3) Anomaly detection

Anomaly detection techniques detect outliers in the unlabeled data under an assumption that most of the data examples are normal, by observing the instances that fit the remainder dataset.

one-class classification

Train a model on only one-class, if anything lay outside of this class, it may be an anomaly.

Algorithms for doing so include...

- (i) one-class k-means
- (ii) one-class SVM
- (iii) Isolation forest

4) Association

we use association algorithm for associating co-occurring items / events.
Associating algorithms

- Apriori



5) Autoencoder (Dimensionality Reduction)

Autoencoders take input data, compress data into a code, then tries to recreate the input data from summarized code.

→ Autoencoder can remove noise from visual data like images, video / medical scans to improve quality.

note

→ Autoencoder can also be used to find outliers (anomaly).

Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

③ Reinforcement Learning

It doesn't require any data or labeled input or labeled output. All it has is the main data.

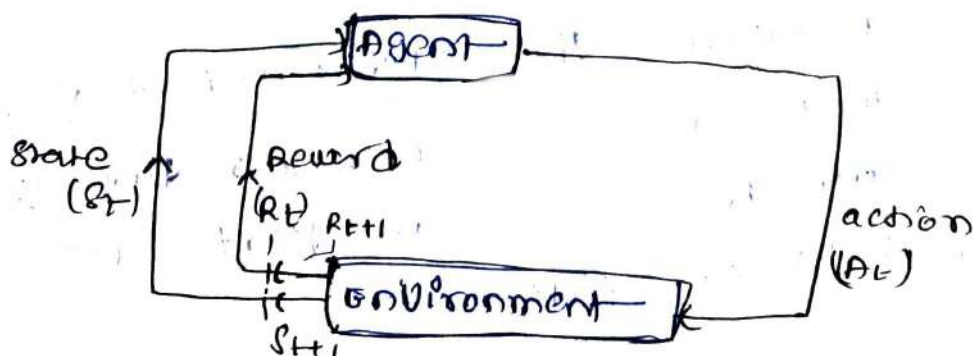
→ like, trial & error method. If it fails in a trial then it learns & tries & it learns etc.

→ Games are a real example of RL.

Take a chess game model. All it has is the environment. First it takes a step & feels the outcome & learns from it and so on.

→ In this kind of ML, AI agents are attempting to find the optimal way to accomplish a particular goal, or improve performance on specific tasks. As the agent takes action that goes toward the goal, it receives a reward.

overall aim Predict & best next step to earn the big reward.

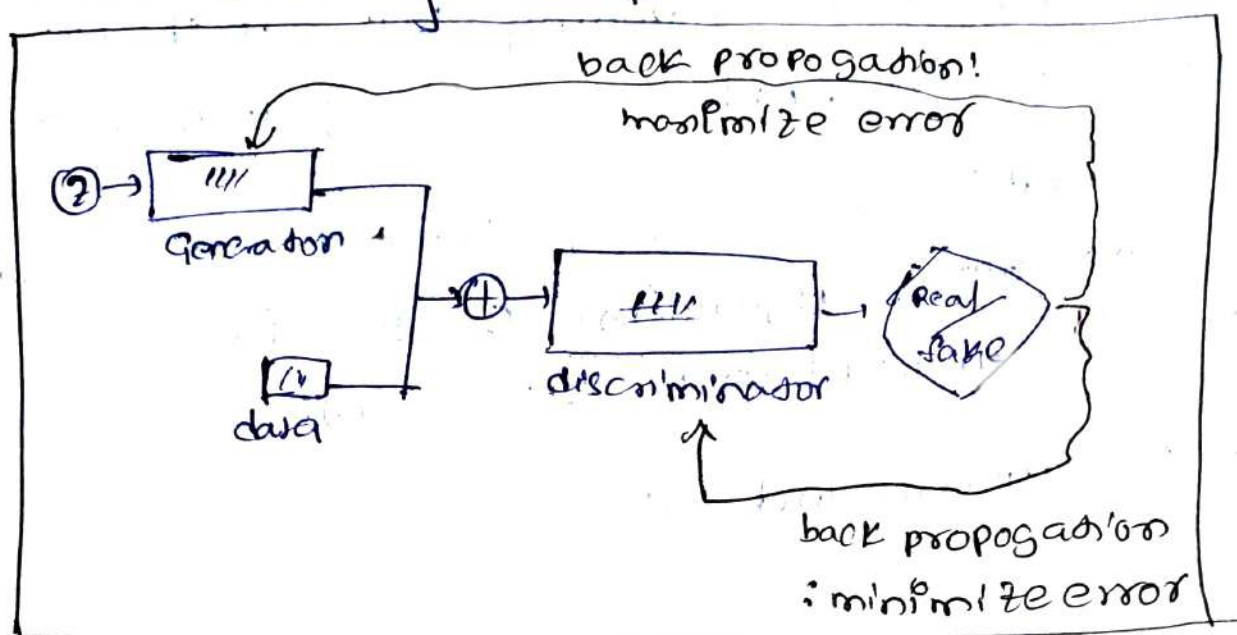


when there is no train, dataset, it learns from experience.

④ Semi-supervised data

- Semi-supervised learning is, for the most part, just what it sounds like: a training dataset with both labeled & unlabeled data.
- This method is particularly useful when extracting relevant features from the data is difficult & labelling examples is a time-intensive task for experts.
- A popular training method that starts with a fairly small set of labelled data is using 'general adversarial networks' / GAN's.
Like two deep learning networks in competition, each trying to outsmart the other. That's a GAN.
- One of the networks called generator, tries to create new data points that mimic the training data.
- The other network the discriminator, pulls in these newly generated data & evaluates whether they are part of training data or fake.

→ the networks improve in a positive feedback loop - as the discriminator gets better at separating the fake from original, the generator improves its ability to create convincing fakes.



egs

→ common situations for this kind of learning are medical images like CTScans (or) MRI's.

A trained radiologist can go through & label a small subset of scans for tumors or diseases.

→ It would be too time-intensive & costly to manually label all the scans - but the deep learning network can still benefit from the small proportion of labeled data & improve its accuracy compared to fully unsupervised models.

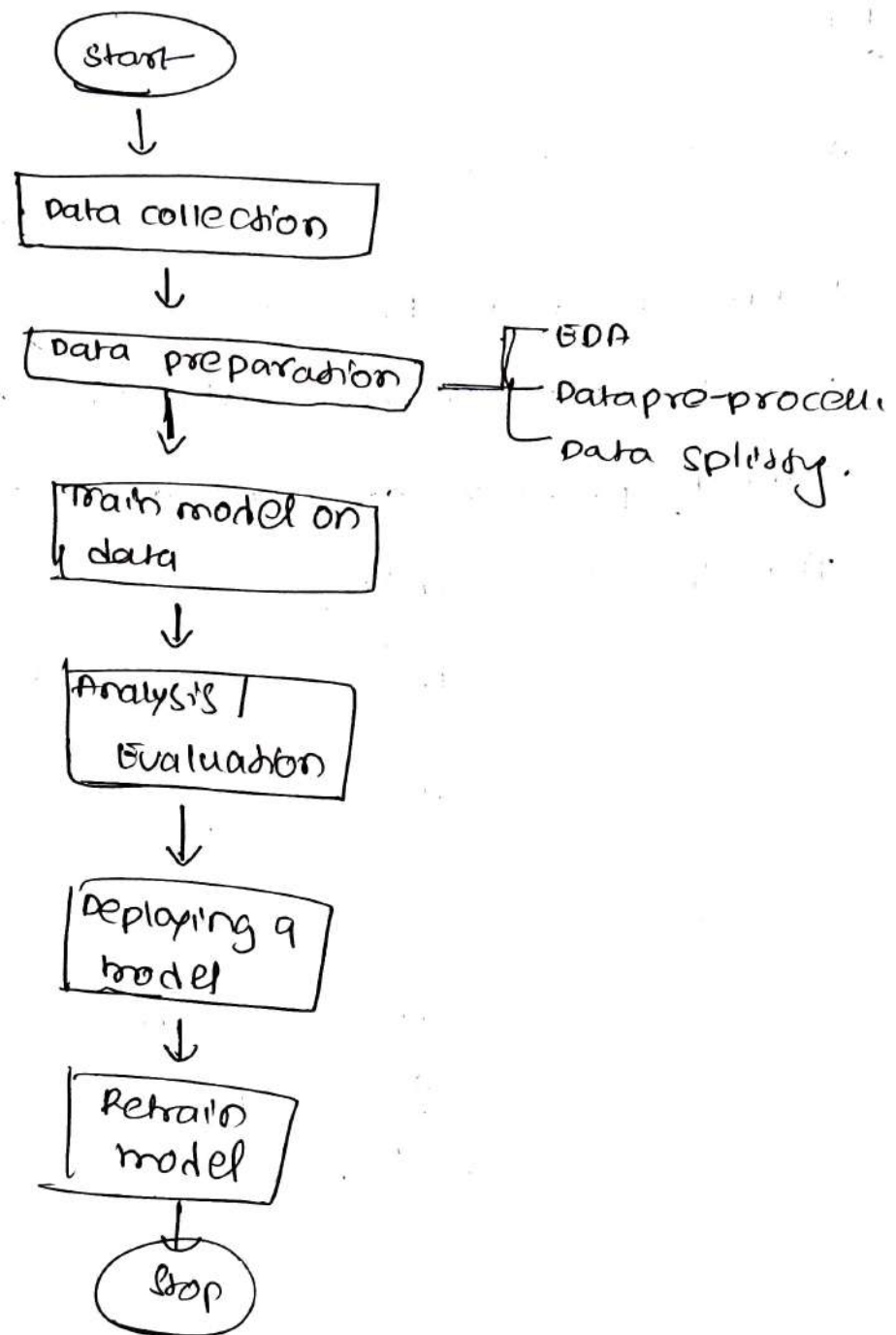
→ It's the basic knowledge about algorithms.
→ Before going deep into algorithms.

Better to know the flow chart of machine learning. I feel ML is like a Christopher Nolan movie, first time you don't understand anything but u should keep watching, at last everything makes sense.

Same works with ML projects, take a small proj on ~~some~~ an algorithm & see it. you will understand the path.

So, let's understand the path.

ML Road map (Steps in machine learning project)



Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

* understanding the process ~-

step ①

Data collection

a) Questions to ask?

- 1) what kind of Problem are we trying to solve?
- 2) what data sources already exist?
- 3) what privacy concerns are there?
- 4) Is the data public?
- 5) where should we store the data?

b) understanding the type of data

1) structured data?

data which appear to be tabulated forms can contain diff types of data.

- wee
- i) nominal / categorical.
 - ii) numerical
 - iii) ordinal
 - iv) time series

2) unstructured data?

data with no rigid structure.

(Image, video, natural language text, speech,

Step 2

Data preparation

It has 3 main processes.

- Exploratory data analysis (EDA), understanding data.
- Data preprocessing, preparing your data to be modeled.
- Data splitting.

a) EDA, understanding the data

- What are feature variables (input) and the target variables (output)?

Ex: For predicting heart disease, feature variable may be a person's age, weight, heart rate etc. and target variable is whether or not they have heart disease.

- What kind of data do you have?
Structured, unstructured, categorical / numerical.
- Are there missing values? Should you remove them / fill them with feature imputation?
- What are outliers?
How many of them are there?
Are they out by much (3+SD)?
Why are they there?
- Are there questions you could ask a domain expert about the data?

b) data preprocessing, preparing your data to be modelled

① Feature Imputation

(Filling missing values)

→ A ML model can't learn on data that isn't there.

- i) Single Imputation: Fill with mean/median of column
- ii) Multiple Imputation: model other missing values and fill with what your model finds.
- iii) KNN (k-nearest neighbour): Fill data with a value from another example which is similar.
- iv) many more such as random imputation, last observation carried forward (for time series), moving window, most frequent.

∴ there are ways to handle missing values.

② Feature encoding

(turning values into numbers)

→ A ML model requires all values to be numeric

i) OneHotEncoding:

Turn all unique values into list of 0's & 1's where the target value is 1 & the rest are 0's.

eg: car colors green, red, blue,

a green car's color feature would be $[1, 0, 0]$

& a red one would be $[0, 1, 0]$

& a blue one $[0, 0, 1]$.

ii) Label Encoder :-

Turn labels into distinct numerical values

eg.

If your target variables are diff

animals as dog, cat, bird then 0, 1, 2 respectively

iii) Embedding encoding

Learn a representation amongst all the diff data points.

eg.

A language model is a representation of how diff words relate to each other. embeddings are also becoming more widely available for structured data.

③ Feature normalization (scaling) or standardization

when your numerical variables are on diff scales. (like size of land b/w 1000 - 2000 sq. ft). In such case some ML models don't perform well. scaling / standardization help to fix this.

i) Feature scaling (normalization)

shifts your values so they always appear b/w 0 & 1.

⊕ This is done by subtracting min value & divide (value b/w 0 & 1 but numerical relationship is still there) with range.

ii) Feature standardization

standardized all values so they have a mean of 0 & variance of 1.

⊕ this is done by subtracting mean & divide with SD.

⊕ values doesn't end up b/w 0 - 1 (at value).

⊕ standardization is more robust to outliers than scaling.

④ Feature Engineering

→ Transform data into potentially more meaningful representations by adding in domain knowledge.

i) Decompose

Like, turn a date (such as 2020-06-18 to 20:16: into hour-of-day, day-of-week, day-of-month, is-holiday etc.

ii) Discretisation

(Turning larger groups to smaller groups)

For numerical variable,

like age you may want to move it into buckets such as over-50 (under-20, 21-30, 31-40 etc), this process is known as BINNING.

For categorical variable,

such as car color, this may mean combining colors such as (light-green, dark-green, lime green) into a single green.

iii) Crossing & Interaction features

(Combining two/more features)

→ depending on situation. Sometimes adding / subtracting two features could help.

iv) Indicator features

like using
is-paid-traffic
for paid

[using other parts of data to
indicate something potentially significant

+ create a x-missing feature for wherever a column x contains a missing value.

⑤ Feature Selection :-

- selecting the most valuable features of your dataset to model.
- Potentially reducing overfitting & training time (less overall data & less redundant data to train on) & improving accuracy.

i) Dimensionality reduction :-

A common dimensionality reduction method, PCA takes a larger no. of dimension (features) & uses linear algebra to reduce them to less dimension. For example, say you have 10 numerical features. You could run PCA to reduce it to 3.

ii) Feature Importance :-

Fit a model to a set of data, then inspect which features were most important to the results, remove the least important ones. (can use "TPOT" for this)

⑥ Dealing with Imbalance :-

Does your data have 10,000 examples of one class, but only 100 examples of another?

- collect more data (if possible)
- use `sklearn-contrib-imbalanced-learn` package.
- use SMOTE (synthetic minority over-sampling technique) which creates synthetic samples of your minor class to try & level the playing field.

→ so this is all about ^{data} Pre-Processing.

c) data splitting

1) training set (usually 90-95% of data):

model, learns on this.

2) validation set (typically 10-15% of data):

model hyperparameters are tuned on this.

3) test set (usually 10-15%): (dev set)

model's final performance is evaluated on this

⊛ do not use this dataset to tune the model

a. why shouldn't we tune on test set?

a.

To do the regularization we have to take 100 diff models & try on the test data. And so last we find (suppose) the best hyperparameters that produce a model with low generalization error, say just 5%.

And you launch it to production & found it produces 15% error.

⊛ now, here is what happened as you trying every hyperparameters on the same test data eventually your model learned the test data & fitted to it thus reducing generalization errors but not improving models.

this is called "overfitting"

→ so to avoid we take validation set /

⊛

cross-validation

Step 3

Train model on dataset

It has 3 main processes.

- a) choose an algorithm
- b) overfit the model
- c) Reduce overfitting with Regularization.

a) Algorithm can be choosed by understanding supervised, unsupervised (explained previous)

b) Types of learning

i) Batch Learning

All of your data exists in a big static warehouse & you train a model on it. You may train a new model once per month once you get new data. Learning may take a while & isn't done often. Runs in production without learning (can be retrained).

ii) Online Learning

Your data is constantly being updated & you constantly train new models on it. Each learning step is usually fast & cheap.

Runs in production & learns continuously.

iii) transfer learning

Take the knowledge one model has learned & use it with your own. Gives you the ability to leverage SOTA (state of the art) models for your own problems. Helpful if you don't have much data / vast compute resources.

iv) Active learning

Also referred to as "human in the loop" learning. A human expert interacts with a model & provides updates to labels for samples which the model is most uncertain about.

v) Ensembling

Not really a form of learning, more combining algorithms which have already learned in some way to get better results.

Underfitting

Happens when your model doesn't perform as well as you'd like on your data. Try training for a longer & more advanced model.

overfitting

Happens when your validation loss (how your model is performing on the validation dataset, lower is better) starts to increase.
or if you don't have a validation set,

* Happens when the model performs far better on the "training set" than on the "test set".

99% accuracy on training set,
67% accuracy on test set → overfitting the training set

Fix through various regularization techniques.

→
Regularization [techniques to prevent/reduce overfitting]

i) L1 (Lasso) & L2 (ridge) regularization

→ L1 regularization sets unneeded feature coefficients to 0.

→ L2 constrains a model's features (won't set them to 0)

ii) Dropout

Randomly remove parts of your model.

So the rest of it has to become better.

iii) Early stopping

Stop your model from training before the validation loss starts to increase too much / more generally, any other metric has stopped improving. Early stopping usually implemented in the form of a model callback.

iv) Data Augmentation

- manipulate your dataset in artificial way to make it harder to learn.
- For example, if you're dealing with images, randomly rotate, skew, flip & adjust the height of your images.
 - this makes your model have to learn similar patterns across diff styles of the same image (harder).
 - use functions like `ImageDataGenerator` in keras or `transforms` in torchvision.

v) Batch Normalization

Standardize inputs as well as adding two parameters (β , how much to offset the parameter for each layer & ϵ to avoid division by zero) before they go into next layer. This often results in faster training speed since the optimizer has less parameters to update, maybe a replacement for dropout in some networks.

Hyperparameter Tuning

→ Run a bunch of experiments with diff model settings & see which works best.

a) setting a learning rate

(often the most important hyperparameter)

Generally

High Learning rate = algorithm rapidly adapts to new data

Low learning rate = algorithm adapts slower to new data
(best for transfer learning)

b) Finding the optimal learning rate

Train the model for a few hundred iterations starting with a very low learning rate ($10e-6$) & slowly increase it to a very large value ($1e-1$) then plot the loss vs learning rate (using a log-scale for learning rate), you should see a U-shaped curve, the optimal learning rate is about 1/2 notch to the left of the bottom of U-curve. (B26-Hands on ML)

ii) Learning rate scheduling involves decreasing the learning rate slowly as model learns more (get closer to convergence). (use Adam optimizer)

iii) cyclic learning rates

Dynamicly change the LR up & down b/w some threshold & potentially speed-up training.

other hyperparameter you can tune

- 1) no. of layers (deep learning networks)
 - 2) Batch size [How many ex of data your model sees at once.]
 - 3) no. of trees (decision tree algo)
 - 4) no. of iterations [How many times model goes through the data]
- ... many more depend on algo using. (can use early-stopping)

Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

Step 4

Analysis/Evaluation

to estimate how well a model will generalize to out-of-sample data.

a) Evaluation metrics

i) Classification

- ROC & AUC curve
- Accuracy
- Precision
- Recall
- F1 score
- Confusion matrix.

ii) Regression

- MSE (mean squared error)
- MAE (mean absolute error)
- R² (r-squared)

task-based metric (create one based on your specific problem).

1) Classification accuracy

Percentage of correct predictions. $\left(\frac{y_{\text{pred}}}{y_{\text{test}}} \right)$

Null accuracy

Accuracy that could be achieved by always predicting the most frequent class.

→ It's a good way to know the min. we should achieve with our model.

→ It gives a baseline to compare our model accuracy.

∴ classification accuracy is easy, but it doesn't tell you the underlying dist. of response class.

→ And it doesn't tell you what type of error your classifier is making.

a) confusion matrix

FP - False Positive
FN - False Negative

	Predicted	
	0	1
Actual 0	TN	FP
Actual 1	FN	TP

TP: which are actually correct

TN: which are actually wrong

they actually don't have corona. But we predict they have.

FP: they are actually wrong, but predicted as true.

they actually have, but we say they don't.

FN: they are actually true, but predicted as false

→ classification

How often is the classifier correct?

$$\text{accuracy} \leftarrow \frac{TP + TN}{TP + TN + FP + FN}$$

talks about correct.

→ misclassification

rate

$$\leftarrow \frac{FP + FN}{TP + TN + FP + FN}$$

Recall/Sensitivity

[True Positive rate]

→ How "sensitive" is the classifier to detecting positive instances?

→ when the actual positive is tested, how often it is correct?

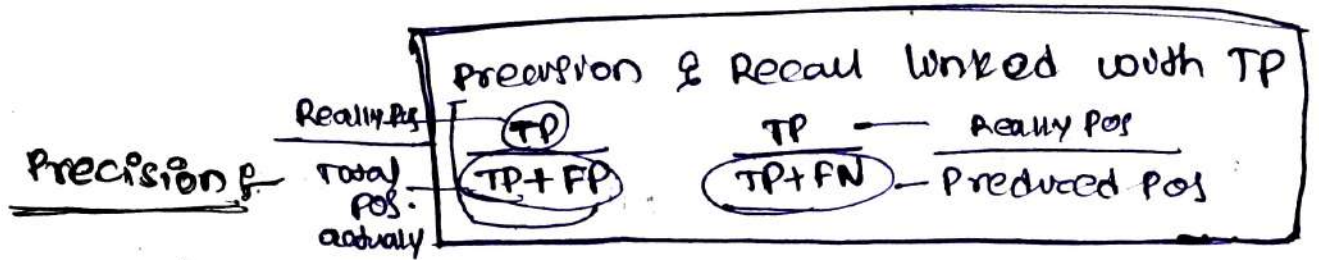
produced pos.

TP

$$\text{Recall} = \frac{TP}{TP + FN}$$

→ It depends on FN, which were actually true, but predicted false.

out of total positive ones, How many positive I am able to Recall



→ when a positive value is predicted, how often is the prediction correct?

→ How "precise" is the classifier when predicting positive instance.

out of Predicted Pos, how many really pos?

$$\text{Precision} = \frac{TP}{TP+FP}$$

→ It depends on FP, which were actually negative, but predicted true.

Q1) which metrics to focus on?

eg 1

1 = COVID 19 (+)
0 = - (Healthy)

	Pred	
Act	0	1
	TN	FP
1	FN	TP

→ Healthy predicted as Sick

→ Sick predicted as healthy

→ now analyze the cost

→ we know a sick predicted as healthy could spread a lot more cases.

So, **cost of FN > cost of FP**

Hence it depends on FN, we go for "Recall".

eg 2

1 = Spam
0 = Not Spam

	Pred	
Act	0	1
	TN	FP
1	FN	TP

→ Not Spam Predicted as Spam

→ Spam got into mail box.

→ we know if a not spam (imp mail) goes to spam, that could cause lot of damage.

So, **cost of FP > cost of FN**

Hence, it depends on FP, we go for "Precision".

Specificity

$$\frac{TN}{TN+PP}$$

False positive rate

$$\frac{PP}{TN+PP}$$

F1-Score

when you want to consider both PP & FN like we need both sensitivity (recall) & precision, in such cases we choose F-Beta.

General Formula

$$F_B = (1+B^2) \cdot \frac{\text{Precision} \times \text{recall}}{(B^2 \cdot \text{Precision}) + \text{recall}}$$

$$F_P = \frac{(1+B^2) \times TP}{(1+B^2) \times TP + B^2 \cdot FN + PP}$$

In beta, there is possibility for 3 ranges.

1) when PP & FN are equally important, like you can't compromise even a bit both are drastic, then we set beta = 1 which is F1-Score.

$$F1\text{-Score} = 2 \times \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

a) when both are worse, but FP is drastic than FN. then we go for $\text{beta} = 0.5$ (usually 0-1)

b) when both are worse, but FN is drastic than FP. then we go for $\text{beta} = 2$ (usually 1-10).

note

we can also adjust classification threshold to modify performance & classify.

Roc curve

can see how specificity & sensitivity are affected by various thresholds, without actually changing the threshold.

AUC curve - (Area under curve)

If you randomly choose one + & one -, AUC represents the likelihood that your classifier will assign a higher predicted probability to the positive observation.

Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

Analysis of model

1) Feature Importance

Which feature contributed most to model
Should some be removed?

2) Training / Inference time

How long does a model take to train?
Is this feasible?

How long does inference take?
Is it suitable for production?

3) using 'what-if' tool

what-if changed sth in data?

How does this affect the outcome?

4) Bias / variance trade-off

→ High bias results in underfitting &
a lack of generalization to new sample

→ High variance results in overfitting due
to the model finding patterns in the
data which is actually random
noise,

Step 5

Deploying a model

Put the model into production & see how it goes. Evaluation metrics in notebook are great but until it's production, you won't know how it performs for real.

Step 6

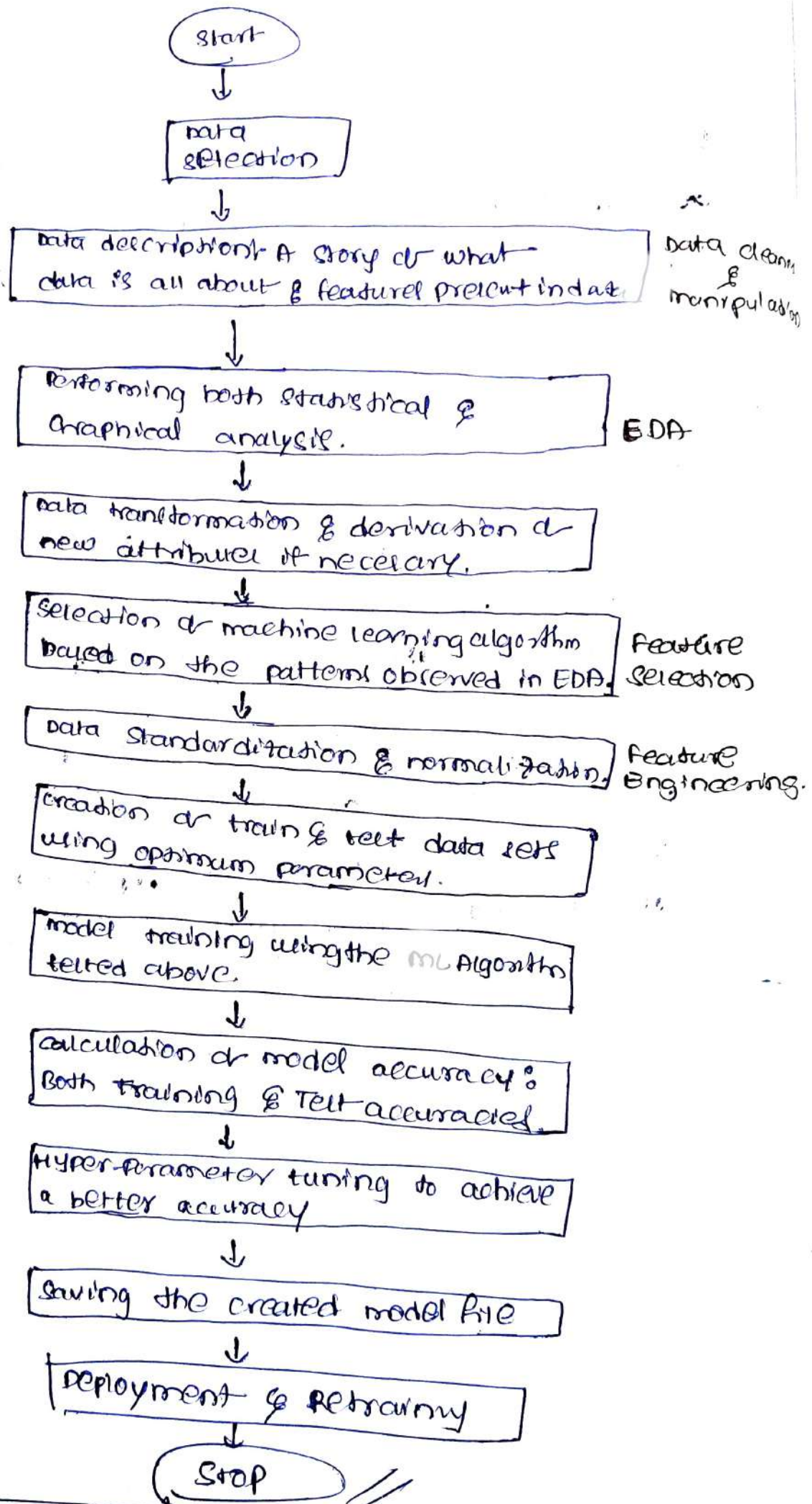
Retrain model

→ see how the model performs after serving based on various evaluation metrics & revisit the above steps as reqd.

→ you will find your model prediction start to age or (drift), as in when data source change or upgrade. that is when you will want to retrain it.

Stop / NO END

So, basically the flowchart is -



Linear Regression

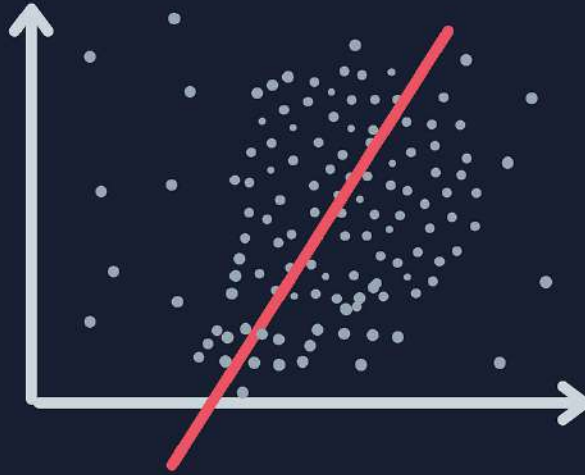


Table of Contents

1. What is Linear Regression
2. Understanding with an example
3. Evaluating the fitness of the model
4. Understanding Gradient descent
5. Understanding Loss Function
6. Measuring Model Strength
7. Another Approach for LR - OLS

Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

understanding the algorithms

① Simple Linear Regression

Linear model in Regression
(supervised learning)

Simple linear regression assumes that a linear relationship exists b/w the response variable & the explanatory variable, it models this relationship with a linear surface called a "hyperplane".

→ This will be clear with an eg.
Let's take the height & weight sample of few people

Height	Weight
5.6	60
5.7	62
5.8	63
5.9	62
6.1	64
6.2	75

$H \times W$
 $W \times H$

Now, if I want to know what would be the weight of a person with 6.0 Height?

→ By observation it seems like

$H \propto W$ → to remove proportionality we can multiply a constant

$W = mH$ → But, it may also differ by some value, so, $\pm b$

$W = mH \pm b$ → now, this is the relation of height & weight.

$w = mH + b$ — constants
 — now, here if I know the value of 'm' & 'b', then I can predict the weights.

Now, main aim is to find out value of 'm' & 'b'.

→ Let's take a sample, (5.6, 60), (5.7, 62),

$$\begin{aligned}
 60 &= m(5.6) + b \quad \text{--- (1)} \\
 62 &= m(5.7) + b \quad \text{--- (2)}
 \end{aligned}
 \left. \vphantom{\begin{aligned} 60 &= m(5.6) + b \\ 62 &= m(5.7) + b \end{aligned}} \right\} \rightarrow \text{By this we get } m=20, b=-52.$$

So, our equation $w = 20H - 52$

Let's try with a known height 5.9.

$$w = 20(5.9) - 52 = 66 \text{ kg.}$$

Here we got '66', but we are expecting 62.

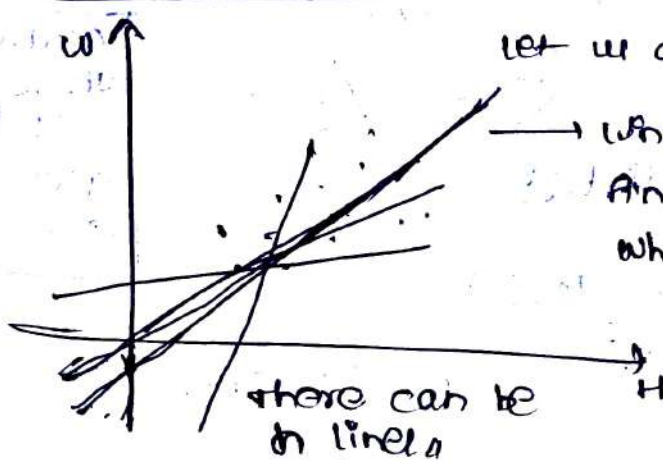
* So, may be if we change ['m' & 'b'] then we could get a exact value / approx.

Now, here we get the definition of ML as establishing a mathematical relationship of data. So, that it can predict a new data.

→ So, when we say a ML model,

* It is nothing but a mathematical relationship, which can predict new data.

Plotting 'H' & 'w'



Let us assume we have a best fit line.

→ Linear regression always tries to find the linear relation b/w variables which is a straight line.

Hence, $y = mx + c$

there can be
 in lines

→ we could find n diff lines.

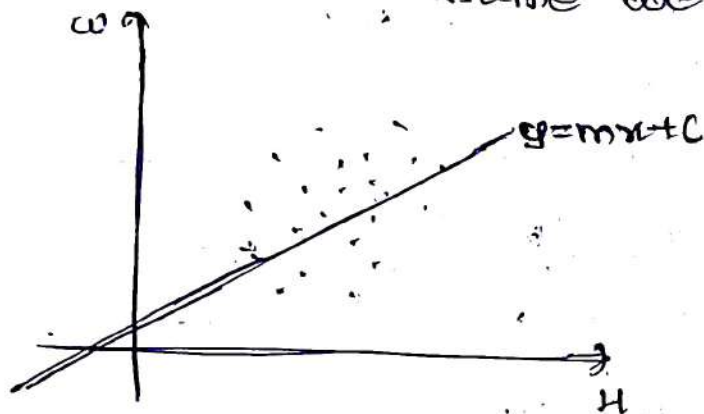
But that one line which can predict all the value approx. is the "Best fit line"

$y = mx + c$ → And this best fit line is called Model
→ we have to find the 'm' & 'c'.

m-slope, c-Intercept

* Evaluating the fitness of the model with a cost function

→ First, let us assume we have a best fit line.



But, now we have doubt.

How did we opt this line?
Why this line?
What about other lines?

→ Now, if it's the best fit line, then for future if I want the weight of '6.0'.

→ It is nothing but the value of y-coordination on $y = mx + c$ line with x-coordination '6.0'.

→ If this line is $w = 20H - 52$ and at $x = 5.9$,

I got $w = 66$, but actually $w = 62$.

So, I can see there is a loss / error.

$$\text{loss} = |(\overset{\text{known}}{y} - \overset{\text{Predicted}}{\hat{y}})|$$

Always there will be a slight error

Q) why did we get this loss?

A) may be best of 'm' & 'c' we taken.

Gradient Descent - 48

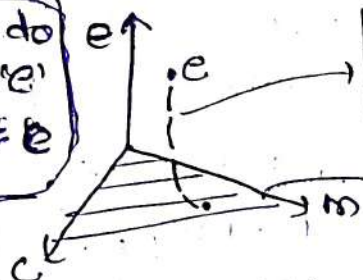
→ Now, to improve my model strength I have to reduce the loss.

and this loss depends on m & c taken.

→ So, I am supposed to shift m & c such that I end up reducing the loss.

we have $\text{loss}(c)$, slope (m), constant (c).

But I have to do is bring these down to m & c plane



should bring down e and find the new coordinate

In this plane we have to error.

→ So, by examining each input m & c get changed and eventually loss decreased. this process is called Learning.

So,

$$m_{\text{new}} = m_{\text{old}} - \eta \Delta E_m$$

$$c_{\text{new}} = c_{\text{old}} - \eta \Delta E_c$$

m_{new} - changed one, new
 m_{old} - old m value.

η - learning rate

$\eta \rightarrow (0.0001 - 10)$ range.

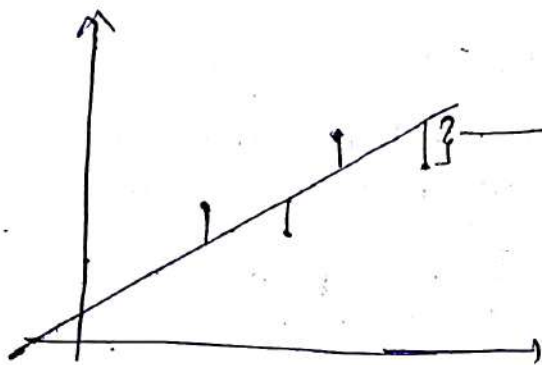
ΔE_m - Error in $m \left(\frac{de}{dm} \right)$

→ So, here we are taking a new m .

By subtracting a fraction of the error rate from old m that fraction is learning rate.

So, now I am supposed to choose the m & c , which gives a least possible error. then it will be the best fit line.

→ A cost function also called a loss func, is used to define & measure the error of a model. the diff. b/w the weights predicted by model & the observed weights in training set are called "residuals" or training error / loss.



this is the residual / loss. which is $|y - \hat{y}|$

→ we have to reduce this residual. Actually, we have to reduce residual of all data points.

→ we can produce best weight predictor model by minimizing the 'sum of residuals'.

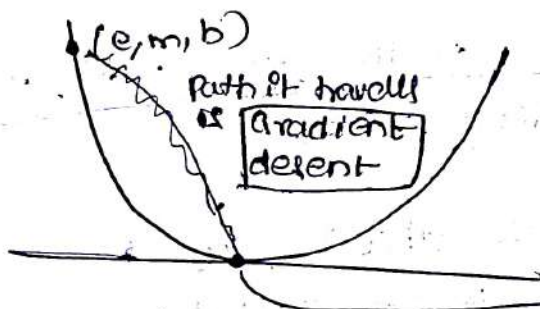
$$(\text{Sum of residuals}) \sum_{i=1}^n r_i = \sum_{i=1}^n [y_i - (mx_i + c)]$$

so, to cancel the negative value, we take square

$$\textcircled{*} R(x) = \sum_{i=1}^n r_i^2 = \sum_{i=1}^n [y_i - (mx_i + c)]^2$$

sum of squares of residuals

now, it is a quadrauve in 3D.



so, we are supposed to shift the (c, m, b) to the origin $(0, m, b)$.

$$\text{where } \frac{dR}{dm} = 0, \frac{dR}{db} = 0.$$

As we can see residual is both a sum of m & b ,
 so, differentiating partially w.r.t m & b will
 give us

$$\frac{\partial R}{\partial m} = \sum_{i=0}^n 2x_i (b + mx_i - y_i)$$

$$\frac{\partial R}{\partial b} = \sum_{i=0}^n 2 (b + mx_i - y_i)$$

so, we know for best line, residual should be min.
 minima of func. occur where derivative = 0.

i.e.,

$$\sum_{i=0}^n 2x_i (b + mx_i - y_i) = 0$$

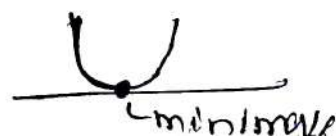
$$\sum_{i=0}^n 2 (b + mx_i - y_i) = 0$$

$$\begin{aligned} \sum_{i=0}^n x_i b + \sum_{i=0}^n mx_i^2 - \sum_{i=0}^n y_i x_i &= 0 \\ \sum_{i=0}^n b + \sum_{i=0}^n mx_i - \sum_{i=0}^n y_i &= 0 \end{aligned}$$

the same eq. can be written in matrix form as

$$\begin{bmatrix} \sum_{i=0}^n x_i^2 & \sum_{i=0}^n x_i \\ n & \sum_{i=0}^n 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} \sum_{i=0}^n x_i y_i \\ \sum_{i=0}^n y_i \end{bmatrix}$$

ideally, if we'd have an eq. of one
 dependent & one independent variable
 the minima will look like



dependent $y = mx + c$ independent var

→ the new values for slope & intercept are calculated.

repeat until convergent

{

$$m_{\text{new}} = m_{\text{old}} - \eta \left[\sum_{i=1}^n (h_0 \cdot x_i - y_i) x_i \right]$$

$\frac{dL}{dm}$

$h_0 = m$

$$c_{\text{new}} = c_{\text{old}} - \eta \left[\sum_{i=1}^n (h_0 \cdot x_i - y_i) \right]$$

$\frac{dL}{dc}$

}

Accuracy this is how LR works.

* measuring model strength

RSS - Residual Sum of Squares
TSS - Total Sum of Squares

The R^2 -squared (R^2) statistic provides a measure of R^2 .

It takes the (form of proportion) - (proportion explained).

$$R^2 = 1 - \frac{RSS}{TSS}$$

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Product
keeps changing

$$TSS = \sum_{i=1}^n (y_i - \bar{y})^2$$

mean.

It $R^2 = 0.75$ it says that our model.

Explains 75% of data set.

→

Adjusted R^2 Statistic

→ As R^2 is just a linear eq., as we increase the no. of independent variables in our eq., the R^2 increases as well.

But, that doesn't mean that the new independent variables have any correlation with the output variable.

i.e., R^2 will increase, but it is not necessary model yields better result.

→ To rectify this, we use adjusted R^2 value which penalises excessive use of such features which do not correlate with the output data.

$$R^2_{adj} = 1 - \frac{(1 - R^2)(N - 1)}{N - P - 1}$$

where,

P - no. of predictors
 N - total sample size.

If $P=0$, $R^2_{adj} = R^2$.

$N - P - 1$

Note:-

→ Using training data to learn the values of the (m & c) parameters for simple linear regression that produce the best fitting model is called ordinary least squares (OLS) (or) linear least square.

Another approach to find $m \& c$.

(to solve OLS)

→ variance is a measure of how far a set of values are spread out.

→ covariance is a measure of how much two variables change together. If the variables increase together, their cov is positive.

0 - no relation, -ve → one increases & one decreases

$$\text{var} = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}$$

$$\text{covar} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

→ variance of explanatory variable.

→ covariance of the response, explanatory variables.

$$\therefore \beta = \frac{\text{cov}(x, y)}{\text{var}(x)}$$

$$y = \beta x + \alpha$$

$$\alpha = \bar{y} - \beta \bar{x}$$

→ we can solve

note

→ The independent variables are uncorrelated with the residual term, also known as "Exogeneity". This in layman term generalises that in no way should the error term be predicted given the value of independent variable.

the error terms have a constant variance.

i.e. Homoscedasticity.

→ the error terms are normally distributed.

→ no multicollinearity, i.e., no independent variables should be correlated with each other or affect another.

→ model confidence

Q1 Is linear regression a low bias / high variance model
(or) a high bias / low variance model?

A It's a "High bias / low variance" model

→ Even after repeated sampling, the fit line won't stay roughly in the same pos. (low variance)

→ But, the avg. of the models created after repeated sampling won't do a great job in capturing the perfect relationship. (High bias)

→ low variance is helpful when we don't have too training data.

```
import statsmodels.formula.api as smf
model_name = smf.ols(formula = ' ~ ', data = )
model_name.conf_int()
```

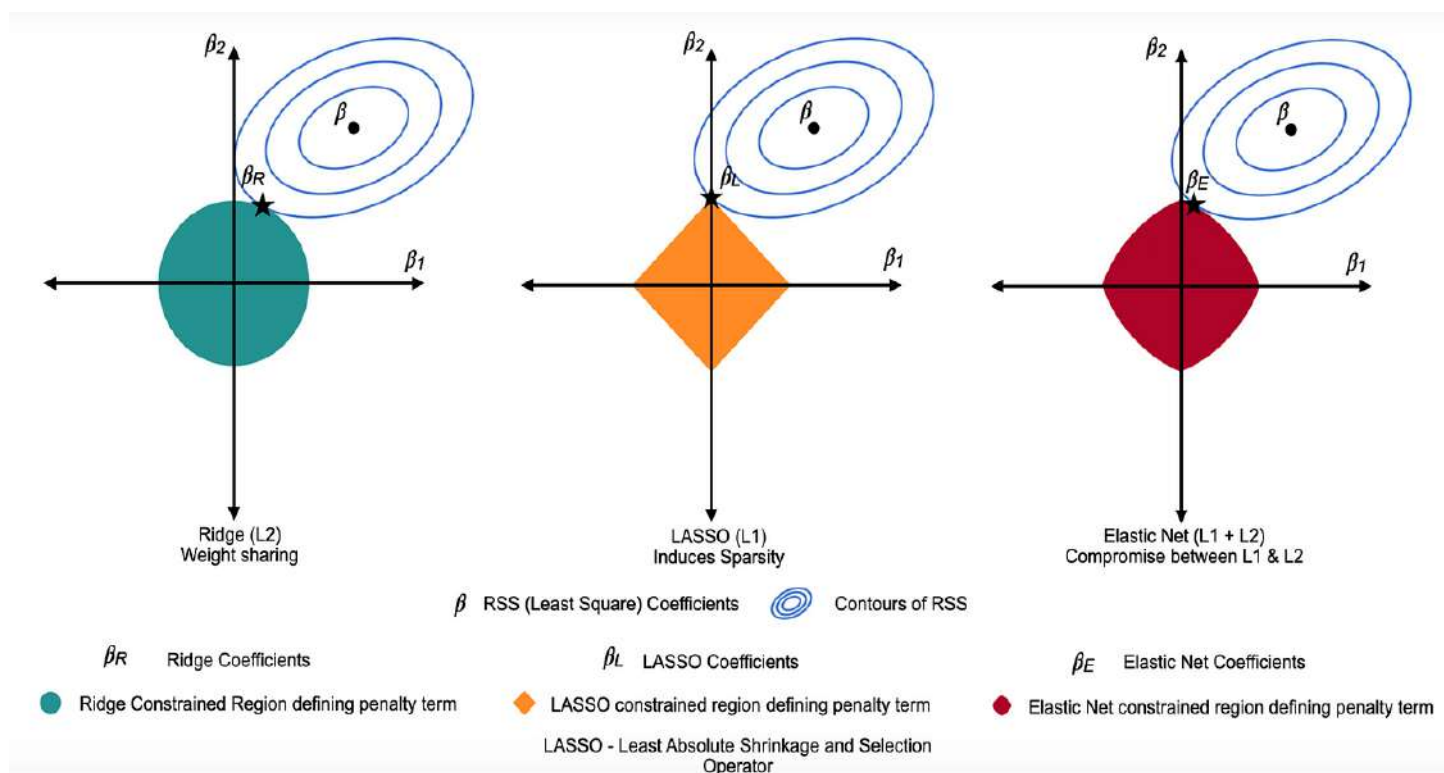



Table Of Contents

1. Understanding Multicollinearity
2. Variance Inflection Factor
3. Regularization
4. Lasso - L1 Form
5. Ridge - L2 Form
6. Elastic Net
7. Difference Between Ridge and Lasso
8. When to use Ridge/Lasso/Elastic Net
9. Polynomial Regression

Join Our Telegram Channel to Learn AI & ML:
<https://lnkd.in/gEpetzaw>

Multi collinearity

→ we can define multi-collinearity as the situation where the independent var have strong correlation among themselves.

→ the coeft. in a linear Reg model represent the extent of change in 'y' when a certain $x (x_1, x_2, x_3, \dots)$ is changed keeping others constant. But if x_1 & x_2 are dependent then this assumption itself is wrong that we are changing one var keeping others constant as the dependent var will also be changed.

It means model becomes a bit flawed.

$$y = mx_1 + nx_2 + c$$

while,

$$x_2 = ax_1 + d$$

Independent var (x_1, x_2) are related.

more than one linear eq,

Hence called

multi collinearity

→ we have redundancy in our model as two variables (or more than two) are trying to convey the same information.

★ → As the extent of collinearity increases, there is a chance that we might produce an "overfitted model".

→ An overfitted model works well with the test data but its accuracy fluctuates when applied to other datasets.

② can result in a dummy variable trap.

→ By the heatmap we can check collinearity.

Generally a correlation greater than 0.9 (or) less than -0.9 are to be avoided.

③ Variance Inflation Factor (VIF) →

Regression of one X var against other X variables.

$$VIF = \frac{1}{1 - R^2}$$

$VIF < 5$ ✓ perfect

→ If $VIF > 5$, extreme correlation is avoided.

Remedies for multi-collinearity

- ① Do nothing :- If corr not extreme / the var not used the ignore
- ② Remove one var like in dummy var trap (one hot encode)
- ③ combine correlated var - combine variables.
- ④ Principle component Analysis (PCA)

* Regularization

→ when we use Regr. models to train some data, there is a good chance that the model will overfit the given training dataset.

→ Regularization helps sort this overfitting problem by restricting the degree of freedom of given eq.

For,

Simply reducing the no. of degree of a polynomial by reducing their corresponding weights.

Reason for Regularization

→ In a linear eq, we don't want huge coeff, as a small change in coeff can make a large diff. So regular constraints weights of such features to avoid overfitting.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

$$RSS = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j \cdot x_{ij} \right)^2 \quad \text{or } (y_i - \hat{y})^2$$

* To Regularize a model, shrinkage penalty is added to cost function.

LASSO (Least Absolute Shrinkage & Selection Operator) (L1 form)

→ LASSO regression penalizes the model based on the sum of magnitude of the coefficients.

The regularization term is

given by, $\boxed{\text{regularization} = \lambda * \sum |B_j|}$

shrinkage factor

→ Loss after regularization is:-

new loss func. $\boxed{RSS + \lambda \cdot \sum_{j=1}^p |B_j| = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p B_j \cdot x_{ij} \right)^2 + \lambda \cdot \sum_{j=1}^p |B_j|}$

Here, Loss is not calculated just by previous values $(y - \hat{y})$, now it's been split into several parts which decreases loss,

Ridge Regression (L2 form)

→ Ridge Regression penalizes the model based on the sum of squares of mag of coeff.

$$\boxed{\text{regularization} = \lambda * \sum B_j^2}$$

λ - can be calculated by cross validation

→ Loss after regularization,

$$\boxed{RSS + \lambda \cdot \sum_{j=1}^p B_j^2 = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p B_j \cdot x_{ij} \right)^2 + \lambda \cdot \sum_{j=1}^p B_j^2}$$

NOTE:-

consider B_1 & B_2 be coeff of LR & $\lambda = 1$.

- ① For LASSO, $\boxed{B_1 + B_2 \leq S}$ → where 'S' is the max value the eq. can achieve. & it
- ② For Ridge, $\boxed{B_1^2 + B_2^2 \leq S}$

Elastic net

→ middle ground b/w Ridge & Lasso

→ the regularization term is a simpler min of both Ridge & Lasso, and you can control min ratio α .

$$L_{\text{enet}}(\hat{\beta}) = \frac{\sum_{i=1}^n (y_i - x_i \hat{\beta})^2}{2n} + \lambda \left[\frac{1-\alpha}{2} \sum_{j=1}^p \hat{\beta}_j^2 + \alpha \sum_{j=1}^p |\hat{\beta}_j| \right]$$

→ where α is the mixing parameter

Ridge ($\alpha=0$)

Lasso ($\alpha=1$) //

(*) difference between Ridge & Lasso

→ Ridge Regression shrinks the coeff for those predictors which contribute very less in the model but have huge weight, very close to 0.

$$\begin{aligned} y &= 0.04n + \text{very less} \\ R^2 &= 0.0016 - \text{very less} \end{aligned}$$

thus final model will still contain all those predictors, though with less weights.

→ where as in Lasso, the L_1 penalty does reduce some coeff exactly to zero, when we use a sufficiently large penalty parameter λ .

so, in addition to regularizing,

Lasso also performs feature selection.

→ why use Regularisation?

- It helps to reduce the variance of the model, without a substantial increase in the bias.
- It reduces variance in the model that means the model won't fit well for dataset other than training data (which is called overfitting).
- the tuning parameter (λ) controls this bias & variance trade off. selected by cross-validation.
- when ' λ ' is increased to certain limit, it reduces the variance without losing any imp prop of data.
- But after certain limit, model will start losing imp prop which will increase bias in the data.

as what should be used plain linear, Ridge, Lasso or Elastic net?

- It is always preferable to have atleast a little bit of Regularisation, so generally avoid plain linear Reg.
- Ridge is a good default,
- But if you suspect that only a few features are actually useful, you should prefer Lasso/EN, since they tend to reduce the useful features weights down to zero.
- In general Elastic net is preferred over Lasso since Lasso may behave erratically when the no. of features is greater than no. of training instances or when several features are strongly correlated.

whenever, mean is not 0,
it's better to do standard scalar

note

① standardScaler() $\uparrow$ $\frac{x - \bar{x}}{SD}$ $\bar{x} = 0$
 $SD = 1$

② Two ways to find multicollinearity — correlation & VIF

③ 4 ways to remove multicollinearity,

④ train set, high accuracy; test set, low accuracy.
i.e. overfitting, so regularize it.

→ Even after checking Lasso and Ridge,
if it gives the same accuracy values,
then you can say it is not over-fitted.

i.e. just by having huge difference in
train accuracy & test accuracy then it
doesn't just mean overfitted. Test before
judging. In this case model is not problem,
more data is needed.

⑤ The lineal Linear Regression does not take
about the degree of polynomial eq. in
terms of dependent var(x).

Instead, it takes about the degree of coeff.

$$y = a + bx + cx^2 + \dots + nx^n$$

It's not taking about power of x,

but the power of a, b, c etc. And their powers.

Hence it is lineal Regression.

Polynomial Regression

→ It is a mechanism to predict a dependent var based on the polynomial relationship with the independent variable.

on the ex, $y = a + bx + cx^2 + \dots + nx^n + \dots$

max. power of x is called degree of polynomial y .

deg ① $\rightarrow y = a + bx$

deg ② $\rightarrow y = a + bx + cx^2$

when to use?

⊕ If data is scatter in a polynomial curve shape (not linear) then keep trying different degree until the line fits to data.

code

```
from sklearn.preprocessing import PolynomialFeatures  
Poly-reg = PolynomialFeatures(degree=2)  
X_Poly = Poly-reg.fit_transform(x)
```

→ keep changing degree until it fits to data

* Logistic regression (is not Regression model)

② Logistic Regression

Linear model for classification
(supervised learning)

* Probability based model

→ Logistic Regression is used for performing 'classification problems'.

→ It calculates the probability that a given value belongs to a specific class. If the probability is more than 50%, it assigns the value to that particular class,

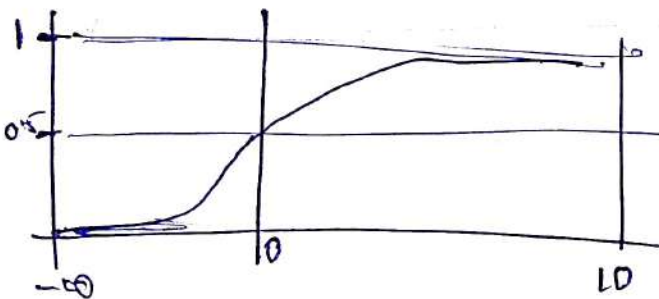
else if probability is less than 50%, the value is assigned to the other class.

∴ we can say that logistic Regr. acts as a binary classifier.

* Here, As we give input x , we get y , where y is the probability of given var x belonging to a certain class.

Thus, it is obvious that the value of y should lie between 0 & 1.

So, we use Sigmoid function as the underlying function in Logistic Regression.



$$\sigma(x) = y = \frac{1}{1 + e^{-x}}$$

Why we sigmoid?

- ① the sigmoid function's range is bounded between 0 & 1, which is what we need.
- ② easy to calculate than other functions which is useful during gradient descent calculation.
- ③ it is a simple way to introduce non-linearity to model

the logistic func is given as

$$P(x) = \frac{e^{B_0 + B_1 x}}{1 + e^{B_0 + B_1 x}}$$

we have $x = B_0 + B_1 x$

$$P(x) = \frac{1}{1 + e^{-(B_0 + B_1 x)}} = \frac{e^{B_0 + B_1 x}}{1 + e^{B_0 + B_1 x}} \quad \text{--- ①}$$

$$1 - P(x) = \frac{1}{1 + \frac{e^{h(\theta)}}{1 + e^{h(\theta)}}} \quad (h(\theta) = B_0 + B_1 x)$$

$$1 - P(x) = \frac{1}{1 + e^{h(\theta)}} \quad \text{--- ②}$$

from divide ①, ②

$$\frac{①}{②} \Rightarrow \frac{P(x)}{1 - P(x)} = e^{h(\theta)} = e^{B_0 + B_1 x}$$

$$B_0 + B_1 x = \log\left(\frac{P(x)}{1 - P(x)}\right) \rightarrow \text{logarithmic function,}$$

Now we have,

$$\log\left(\frac{p(n)}{1-p(n)}\right) = \beta_0 + \beta_1 x$$

(logit function)

As we input x , we will get $p(n)$ (probability)

→ And we get $p(n)$, based on the line with β_0, β_1 constants.

this is how logistic regression works

well, if we have two classes (0+1)

$$y = \begin{cases} 0 & , \text{ if } p(n) < 0.5 \\ 1 & , \text{ if } p(n) \geq 0.5 \end{cases}$$

0.5-threshold

→ Based on the scenario

we should change threshold

we can change threshold

Cost Function / Loss Function

for a single training instance,

$$\text{cost}(p(n), y) = \begin{cases} -\log(p(n)) & , \text{ if } y=1 \\ -\log(1-p(n)) & , \text{ if } y=0 \end{cases}$$

if $p(n)=1$ & $y=1$, cost/loss = 0

if $p(n)=0$ & $y=0$, cost/loss = 0

But,

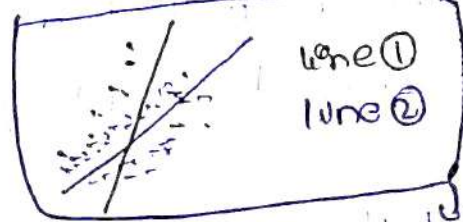
if $p(n)=0$ & $y=1$, cost/loss $\rightarrow \infty$

if $p(n)=1$ & $y=0$, cost/loss $\rightarrow \infty$

a cost func for whole training set is given as

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \cdot \log(\hat{p}_i) + (1-y_i) \cdot \log(1-\hat{p}_i) \right]$$

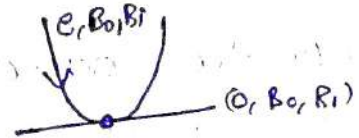
where, m - rows
 y_i - expected y
 $\hat{p}_i$ - predicted y_i



On logit func, RHS we have a line, and we will keep on adjusting the line w.r.t error so that will get correct probability, for given data

→ the value of parameter (θ) for which the cost func is min is calculated using the gradient descent algorithm (as shown in 11).
 the partial derivative for above cost func is,

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left[\sigma(\theta^T x_i) - y_i \right] \cdot x_i$$



multinomial Logistic Regression

→ we can extend logistic regression for multi-class classification. the logic is, we train our model for each class and calculate the probability that a specific feature belongs to that class. once we have trained the model for all the classes, we predict a new values class by choosing that class for which $\text{prob}(\text{h}(x))$ is maximum.

* we rarely use it this way, because we have many other classification models for these scenarios